Ejercicio 17: cantidad de operaciones y uso de memoria

Para analizar el costo del procedimiento, es necesario distinguir entre la generación de los puntos equiespaciados y la evaluación de los tres polinomios.

Generación de los puntos

Para generar 100 puntos equiespaciados entre -1 y 1, se define primero la distancia entre dos puntos consecutivos:

$$h = \frac{2}{100 - 1} = \frac{2}{99}.$$

Luego, cada punto se calcula mediante:

$$x_i = -1 + ih, \qquad i = 0, \ldots, 99.$$

Esto requiere:

- una división para calcular h;
- 100 multiplicaciones;
- 100 sumas.

Por lo tanto, la cantidad total de operaciones aritméticas necesarias para generar los puntos es:

$$\boxed{201 \text{ operaciones}}.$$

Si cada punto se almacena como un número de punto flotante de 64 bits, cada uno ocupa 8 bytes. Por lo tanto, los 100 puntos ocupan:

$$100 \cdot 8 = \boxed{800 \text{ bytes}}.$$

Este cálculo considera únicamente el espacio ocupado por los datos del arreglo y no incluye la sobrecarga interna introducida por las listas y los objetos de Python.

Evaluación de los polinomios por bloques

El tamaño de bloque utilizado es 11, debido a que el polinomio de mayor grado tiene grado 10. Para procesar los 100 puntos se necesitan:

$$\left\lceil \frac{100}{11} \right\rceil = 10$$

bloques.
Para cada bloque y cada polinomio se realizan las siguientes operaciones:

- La construcción de una matriz de Vandermonde de tamaño 11×11 requiere 121 potencias.
- La multiplicación de la matriz por el vector de coeficientes requiere 121 multiplicaciones y 121 sumas.

Por lo tanto, para cada bloque y cada polinomio se realizan:

$$121 + 121 + 121 = 363$$

operaciones.
Como se procesan 10 bloques para cada uno de los 3 polinomios, la cantidad total de operaciones para evaluarlos es:

$$363 \cdot 10 \cdot 3 = \boxed{10890 \text{ operaciones}}.$$

Al sumar las 201 operaciones correspondientes a la generación de los puntos, se obtiene:

$$10890 + 201 = \boxed{11091 \text{ operaciones}}.$$

En este conteo, cada potencia se considera como una única operación. Si cada potencia se descompusiera en multiplicaciones elementales, la cantidad de operaciones sería mayor.

Espacio en memoria

Considerando únicamente el espacio ocupado por los arreglos numéricos principales:

- Los 100 puntos ocupan:
$$100 \cdot 8 = 800 \text{ bytes}.$$
- Los resultados de los tres polinomios ocupan:
$$3 \cdot 100 \cdot 8 = 2400 \text{ bytes}.$$
- La matriz de Vandermonde de 11×11 ocupa:
$$11 \cdot 11 \cdot 8 = 968 \text{ bytes}.$$
- La matriz traspuesta ocupa otros:
$$968 \text{ bytes}.$$
- El vector de coeficientes y el vector resultante del bloque ocupan:
$$2 \cdot 11 \cdot 8 = 176 \text{ bytes}.$$

En consecuencia, el espacio aproximado ocupado por los arreglos principales es:

$$800 + 2400 + 968 + 968 + 176 = \boxed{5312 \text{ bytes}}.$$

Esto equivale aproximadamente a:

$$\boxed{5{,}2 \text{ KiB}}.$$

Este valor no incluye la memoria adicional utilizada por las listas de Python, los objetos internos ni las estructuras creadas para realizar el gráfico.

Crecimiento con respecto a nn

Si se construyera una única matriz de Vandermonde de tamaño $n \times n$, tanto la cantidad de operaciones como el espacio en memoria crecerían cuadráticamente:

$$\boxed{\text{Operaciones} = \Theta(n^2)}$$

y

$$\boxed{\text{Memoria} = \Theta(n^2)}.$$

En cambio, al trabajar con bloques de tamaño fijo 11, la cantidad de bloques es proporcional a n. Por lo tanto, la cantidad de operaciones crece linealmente:

$$\boxed{\text{Operaciones} = \Theta(n)}.$$

El espacio auxiliar ocupado por la matriz de Vandermonde y su traspuesta permanece constante, ya que ambas tienen un tamaño máximo de 11×11. Sin embargo, almacenar todos los puntos y todos los resultados requiere una cantidad de memoria proporcional a n:

$$\boxed{\text{Memoria total} = \Theta(n)}.$$

Modificación para mejorar la eficiencia

Para hacer el cálculo más eficiente, se podría modificar el orden de los ciclos. La matriz de Vandermonde depende únicamente de los puntos del bloque y no de los coeficientes del polinomio.

Por lo tanto, para cada bloque se puede calcular una sola vez la matriz de Vandermonde y su traspuesta:

$$V = \text{matrizVandermonde(bloque)}, \qquad V^T = \text{traspuesta}(V).$$

Luego, la misma matriz traspuesta puede reutilizarse para evaluar los tres polinomios, cambiando solamente el vector de coeficientes.

De esta forma, se evita construir y transponer tres veces la misma matriz para cada bloque. La complejidad asintótica continúa siendo lineal con respecto a n, pero se reduce la cantidad concreta de operaciones realizadas y también la creación innecesaria de matrices temporales.